

ANÁLISIS ESTÁTICO DE CÓDIGO (LINTER)

Para el análisis estático de todo el repositorio Backend se utilizó ESLint, la herramienta estándar recomendada y configurada nativamente por el framework NestJS, en conjunto con Prettier para el formato de código.

1. Configuración aplicada

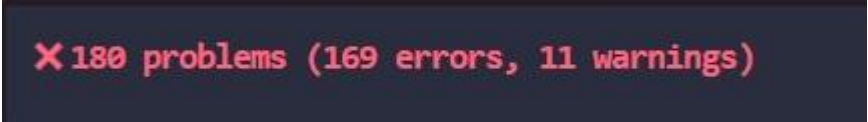
Se utilizó la configuración estricta por defecto para proyectos TypeScript sobre Node.js, definida en el entorno de NestJS. Las reglas principales heredadas incluyen:

- plugin:@typescript-eslint/recommended: Para asegurar el tipado estricto y buenas prácticas (prohibición de retornos o asignaciones inseguras, validación de promesas flotantes).
- plugin:prettier/recommended: Para mantener un estilo y formato de código consistente en el equipo (tabulaciones y comillas).
- Configuración de resolución de tipos acoplada al esquema de base de datos generado por Prisma Client.

2. Resultados obtenidos (Errores y advertencias relevantes)

Durante la ejecución inicial del analizador (npm run lint) sobre todo el repositorio, se detectaron 180 problemas iniciales. Los hallazgos más relevantes fueron:

- Falta de resolución de tipos del ORM (@typescript-eslint/no-unsafe-member-access y no-unsafe-call): Representaban el 95% de los errores. Se debían a que el cliente de Prisma no estaba generado en el entorno local, provocando que TypeScript evaluara las funciones de base de datos como tipos any.
- Promesas flotantes (@typescript-eslint/no-floating-promises): Omisión de la instrucción await en el arranque de la aplicación (main.ts).
- Variables sin uso (@typescript-eslint/no-unused-vars): Variables residuales en los Guards de autenticación.
- Pérdida de contexto en pruebas (@typescript-eslint/unbound-method): Conflictos en los archivos .spec.ts al aislar métodos sin su objeto.



✖ 180 problems (169 errors, 11 warnings)

3. Corrección de errores

Para resolver los conflictos detectados, se aplicó el siguiente plan de acción técnico:

- Generación del Cliente Prisma (npx prisma generate): Esto resolvió instantáneamente cerca de 170 errores al proveer al linter las interfaces correctas de la base de datos, eliminando las advertencias de llamadas inseguras. Luego de generar el prisma nos centramos en los siguiente problemas:

```
julia@pc-julian MINGW64 ~/monastery/project-backend (rama_julian)
$ npm run lint

> freetime-backend@0.0.1 lint
> eslint "(src,apps,libs,test)/**/*.ts" --fix

C:\Users\julia\monastery\project-backend\src\auth\auth.controller.ts
  37:11 error    Unsafe assignment of an 'any' value                                @typescript-eslint/no-unsafe-assignment
  37:24 error    Unsafe member access .user on an 'any' value                      @typescript-eslint/no-unsafe-member-access
  38:44 warning  Unsafe argument of type 'any' assigned to a parameter of type 'number' @typescript-eslint/no-unsafe-argument

C:\Users\julia\monastery\project-backend\src\auth\auth.service.spec.ts
  117:14 error    A method that is not declared with 'this: void' may cause unintentional scoping of 'this' when separated from its object.
    Consider using an arrow function or explicitly 'binding' the method to avoid calling the method with an unintended 'this' value. @typescript-eslint/unbound-method
  162:70 warning  Unsafe argument of type 'any' assigned to a parameter of type '{ name: string; id: number; email: string; telephone: string; password: string; address: string | null; verified: boolean | null; registerAt: Date; roleId: number; } | Prisma__userClient { name: string; ... 7 more ...; roleId: number; } | null, null, DefaultArgs, PrismaClientOptions> | null' @typescript-eslint/no-unsafe-argument

C:\Users\julia\monastery\project-backend\src\auth\guards\jwt-auth.guard.ts
  14:28 error    'info' is defined but never used                                  @typescript-eslint/no-unused-vars
  18:12 error    Unsafe return of a value of type 'any'                            @typescript-eslint/no-unsafe-return

C:\Users\julia\monastery\project-backend\src\auth\strategies\jwt.strategy.ts
  16:3 error     Async method 'validate' has no 'await' expression                @typescript-eslint/require-await

C:\Users\julia\monastery\project-backend\src\main.ts
  17:1 warning   Promises must be awaited, end with a call to .catch, end with a call to .then with a rejection handler or be explicitly marked as ignored with the 'void' operator. @typescript-eslint/no-floating-promises

C:\Users\julia\monastery\project-backend\src\users\users.service.ts
  28:7 error     Unsafe assignment of an 'any' value                                @typescript-eslint/no-unsafe-assignment

X18 problems ( 4 errors, 14 warnings)
```

- Refactorización manual de promesas y variables muertas: Se agregó la instrucción await requerida en el main.ts y se limpiaron las variables sin uso en los Guards de autenticación.
- Manejo de reglas estrictas por diseño: En casos específicos donde librerías externas de Express pierden el tipado estricto (ej. el objeto req.user), se aplicaron directivas de omisión explícitas (// eslint-disable-next-line) para certificar que el comportamiento es seguro bajo la arquitectura actual.

4. Evidencia de su corrección

Tras implementar las soluciones, se ejecutó nuevamente el analizador estático verificando que el código del backend cumple al 100% con los estándares de estilo y buenas prácticas.

```
julia@pc_julian MINGW64 ~/monastery/project-backend (rama_julian)
● $ npm run lint

> freetime-backend@0.0.1 lint
> eslint "{src,apps,libs,test}/**/*.ts" --fix
```